

University of London
BSc Computer Science

AI Study Companion

An inspectable, local-first workflow
for mixed study materials

CM3070 Computer Science Final Project
Final report

Lukas Zemolochinas
Student number 230745052

26 September 2026

Template: CM3020 Artificial Intelligence, section 4.1, Project Idea 1
Orchestrating AI models to achieve a goal

Total chapter word count: 9,842 / 10,500

Counts use Google Docs selections of each complete chapter, including chapter titles, subsection headings, table text and captions. Cover, contents, references and text within images are excluded. This conservative total includes material exempted by the course guidance.

Contents

1 Introduction (841/1,000 words)	3
1.1 Project and motivation	3
1.2 Intended users and boundaries	3
1.3 Objectives and research questions	4
1.4 Final scope and contribution	4
2 Literature Review (2,092/2,500 words)	5
2.1 Review approach	5
2.2 Retrieval and the meaning of evidence	6
2.3 Lexical retrieval, fusion and diversity	6
2.4 Embeddings and model selection	7
2.5 Speech and visual inputs	7
2.6 Learning activities and interface design	8
2.7 Synthesis and contribution	9
3 Design (1,843/2,000 words)	10
3.1 Requirements and evidence	10
3.2 Architecture and deployment	11
3.3 Data representation and provenance	12
3.4 Control flow and failure policy	13
3.5 Interaction design	15
3.6 Privacy and trust boundaries	16
3.7 Alternatives and development decisions	16
4 Implementation (2,242/2,500 words)	17
4.1 Organization and reproducibility	17
4.2 Ingestion and model adapters	17
4.3 Preprocessing and source balance	18
4.4 Retrieval and summary construction	18
4.5 Cloud generation and validation	19
4.6 Questions and persistent practice	20
4.7 Traceability, search and visualization	21
4.8 Interface engineering and release controls	22
5 Evaluation (2,414/2,500 words)	24
5.1 Questions and method	24
5.2 Automated and browser results	24
5.3 Offline engine comparison	26
5.4 Model integration and observed failures	28
5.5 Informal feedback and design changes	28
5.6 Critical assessment and validity	29
6 Conclusion (410/1,000 words)	31
6.1 Outcome	31
6.2 What the evidence supports	31
6.3 Priorities for further work	31
References	32

1 Introduction (841/1,000 words)

1.1 Project and motivation

AI Study Companion is a browser application that turns a learner's notes, recordings, transcripts and images into a revision pack. The pack contains selected passages, concepts, questions, flashcards and links back to the material used. The project follows CM3020 Artificial Intelligence, section 4.1, Project Idea 1, Orchestrating AI models to achieve a goal, within the CM3070 Computer Science Final Project [1]. Its goal is a complete study workflow assembled from several pretrained models and conventional information-retrieval methods, rather than a newly trained model.

The starting problem is practical. A learner may have a reading in PDF format, a spoken explanation and a diagram, but no convenient way to review them together. Transcribing or summarizing each item separately does not resolve the whole task. The learner must still identify useful material, construct questions and check whether an answer reflects the original source. An application that hides these intermediate steps may save effort while making mistakes harder to notice.

I chose to make the evidence inspectable. Each study item refers to a retained source sentence, and the Sources view exposes material that was read but not selected. This makes omission visible as well as selection. The contribution is therefore a working orchestration and review system: mixed inputs share one representation, alternative engines operate on that representation, and the learner can move from an answer back to its evidence without leaving the workspace. Traceability is an engineering property, however, not proof that the evidence or answer is correct.

1.2 Intended users and boundaries

The primary users are adult university students studying independently, particularly those working with English-language readings and recorded explanations. This is a design focus, not a claim that I surveyed a representative student population. I developed the requirements from the task analysis, relevant literature and informal feedback from my mother, father, adult sibling and a software-developer friend. Chapter 5 distinguishes that feedback from instrumented testing and explains its limits.

A secondary use emerged when a family member reviewed meeting transcripts through summaries and questions. This suggests that the workflow may help with recalling non-course material, but it does not establish a second validated market. I do not disclose those transcripts or workplace details. I describe my sibling's school materials only at the level needed to explain the feedback.

The application is not an assessment marker, a replacement for course teaching or a general academic-writing assistant. It cannot guarantee higher grades. Its local engines select and organize source material; only the optional cloud engine writes new summary and question text. This distinction prevents the presence of AI-related components from being mistaken

for a language model in every processing path. English-only speech recognition and uneven device capabilities further constrain the intended use.

1.3 Objectives and research questions

The central question is whether a browser can coordinate mixed-input processing into a usable, inspectable revision workflow while preserving useful material when individual components fail. Four objectives make that question testable.

O1 is to accept supported text, audio and image inputs, normalize them into source records and retain valid material after recoverable file failures. O2 is to produce structured study outputs with resolvable evidence identifiers, visible coverage and exportable results. O3 is to support a coherent learner workflow across reading, practice, source inspection and revision of inputs. O4 is to compare a simple keyword baseline with a more selective retrieval pipeline and document the costs and limitations of model-backed alternatives.

These objectives separate three questions that can otherwise become confused. Can the software complete a task? Can a person inspect its output? Does using it improve learning? This project provides substantial evidence for the first question, limited formative evidence for the second and no controlled evidence for the third. A successful build or a high source-link percentage cannot answer all three.

1.4 Final scope and contribution

The final implementation is a static HTML, CSS and JavaScript application built with Vite and deployed through GitHub Pages. Processing uses disposable Web Workers, browser document parsers, local Whisper speech recognition, local OCR and vision models, optional MiniLM embeddings and an explicitly selected Google generation service. Four engines expose the trade-off between lightweight extraction, more selective retrieval, semantic similarity and cloud-written output. A shared revision-pack schema keeps these alternatives compatible with the same interface.

I evaluate the September release of AI Study Companion [20]. The latest check passed 255 automated tests in 54 files and a production build. A five-case, two-engine benchmark completed 30 runs without structural-integrity failures. These are bounded software results. They do not make the application bug-free or establish educational effectiveness.

The public repository is <https://github.com/LukoDevelops/AIStudyCompanion> and the demonstration is <https://lukodevelops.github.io/AIStudyCompanion/>. The remaining chapters explain the research basis, design decisions, implementation and evaluation before drawing conclusions. I led the project and developed the application, reviewing and integrating AI-assisted work where useful. AI-assisted activities included prototyping, scaffolding, debugging, automated QA and test-case development. Pretrained models and third-party libraries are identified separately from the application's own logic.

2 Literature Review (2,092/2,500 words)

2.1 Review approach

The review addresses a system-design question: which existing techniques help transform mixed study inputs into material that a learner can inspect and practice? It covers retrieval and evidence attribution, speech and image processing, learning activities, and interface evaluation. Research papers are used for methodological claims, while model cards and software documentation identify the particular components deployed. This is a targeted engineering review, not a systematic review with an exhaustive search protocol.

The earlier report established that the components were plausible but gave too little comparison of evidence. The important distinction here is between a study's task, its measurement method and the decision that can reasonably transfer to this project. Open-domain question answering, image captioning and delayed recall measure different outcomes. Their scores cannot be combined into a single measure of study-tool quality. Table 1 summarizes the resulting design choices and the limits of that transfer.

Table 1. Research evidence and its application to this project.

Research area	Evidence considered	Design consequence	Limit on inference
Retrieval and attribution	RAG, ALCE, BM25 and rank fusion [2-5]	Retrieve passages and retain explicit source identifiers	A valid identifier does not establish entailment
Diversity	MMR [6]	Penalize redundant selections	Diversity can reduce focus if weighted poorly
Semantic representations	Sentence-BERT and MiniLM model card [7,8]	Optional sentence embeddings	Similarity benchmarks do not establish study usefulness
Speech and vision	Whisper, BLIP and Florence-2 [9-11]	Separate, reviewable extraction adapters	Published model scores are not browser accuracy estimates
Learning and presentation	Retrieval practice and multimedia learning [12,13]	Questions, feedback and separated views	No learning gain follows without a learner study

2.2 Retrieval and the meaning of evidence

Lewis et al. [2] combine a sequence-to-sequence generator with a dense index of Wikipedia passages. They compare retrieval conditioned at sequence and token level, and report state-of-the-art results on three open-domain question-answering tasks. This supports the value of access to external information over reliance on model parameters alone. However, their learned retriever and large corpus differ from this project's small collection of user-provided sentences. The application adopts the principle of generation conditioned on selected evidence, not the paper's training recipe or reported performance.

Gao et al. [3] make a more directly useful distinction through ALCE: fluency, correctness and citation quality are separate evaluation dimensions. Their benchmark combines question sets and retrieval corpora; even the best evaluated models lacked complete citation support about half the time on ELI5. That finding challenges the assumption that adding citations solves hallucination. Here, an evidence identifier must resolve and a quoted span must match its source. Those checks are necessary for navigation and audit, but they do not establish that the cited passage supports the complete generated claim.

The two papers therefore inform different layers. Retrieval supplies candidate evidence; attribution evaluation asks whether the answer uses it properly. The local modes avoid some generation risk by copying selected sentences, whereas the cloud mode can paraphrase and introduce additional reasoning. A copied sentence can still be misleading when removed from context, and an accurately transcribed source can still contain an error. For this reason, the interface presents source relevance as a review aid rather than a probability of truth.

2.3 Lexical retrieval, fusion and diversity

Robertson and Zaragoza [4] explain the probabilistic relevance framework underlying BM25. Term-frequency saturation and document-length normalization provide a more useful lexical baseline than simply counting repeated words. In this application the retrieval units are sentences, so the original document-retrieval assumptions are only approximate. Short sentences and repeated technical phrases can dominate the ranking even when they contribute little explanation. BM25 is valuable because its behavior is inexpensive and inspectable, not because it solves importance estimation.

Cormack, Clarke and Buettcher [5] evaluate reciprocal rank fusion on TREC runs and LETOR ranking results. Fusion combines rank positions rather than assuming that different systems' raw scores share a scale. Their experiments generally improve on individual rankers, with an important exception involving a human-assisted TREC result. This motivates combining lexical signals, but not claiming that fusion must improve every study pack. The implementation uses the conventional constant 60; it has not independently optimized that value for lecture notes.

Carbonell and Goldstein [6] propose maximal marginal relevance to balance relevance with novelty. This is a better match for a short revision pack than returning many near-identical high-scoring sentences. The project uses an incremental redundancy penalty when selecting additional passages. A risk remains: diversity can promote a less central sentence, and forcing source representation can include weak material. The evaluation therefore examines

how many distinct passages support questions rather than assuming that more questions or more sources always mean better output.

These methods are complementary. BM25 estimates lexical match, fusion reconciles competing rankings, and MMR changes the composition of the selected set. Their purposes should not be collapsed into the vague description of an advanced AI algorithm. They also create a useful ablation question for future work: how much of the observed change comes from retrieval, source quotas or question-generation rules? The current baseline comparison changes several components together and cannot isolate each contribution.

2.4 Embeddings and model selection

Reimers and Gurevych [7] address the cost of sentence-pair comparison by producing independently reusable sentence embeddings. Their Sentence-BERT evaluation covers semantic textual similarity and transfer tasks. The reported reduction from roughly 65 hours to about five seconds for a particular 10,000-sentence similarity search illustrates an architectural advantage, not an expected runtime on this application or device. Reusable representations are relevant because study material is queried repeatedly during summary and concept construction.

The selected all-MiniLM-L6-v2 model maps text into 384-dimensional embeddings and is intended for tasks such as semantic search; its model card describes contrastive training on sentence pairs and default truncation beyond 256 word pieces [8]. The deployed Xenova conversion runs through Transformers.js. Short sentence records fit this use better than embedding an entire reading as one item. Nevertheless, encoding sentences separately can lose pronoun context, and semantic proximity can blur opposites or numerical distinctions. Lexical retrieval remains part of the pipeline rather than being replaced wholesale.

The current Semantic Explorer compares candidate sentences with selected anchor sentences. It is not an unrestricted dense search across a large persistent vector database. This bounded design reduces repeated similarity work, but it also constrains discovery: a passage excluded by lexical candidate selection cannot be rescued later by embeddings. The project retains the inexpensive Hybrid Focus engine as the default because the available evidence does not establish that model downloads and extra computation consistently improve study quality.

2.5 Speech and visual inputs

Radford et al. [9] train Whisper using 680,000 hours of multilingual, multitask supervision and report strong generalization across speech benchmarks without dataset-specific fine-tuning. This provides a credible basis for using a pretrained speech adapter. The application deploys quantized whisper-base.en, an English model, rather than the largest model from the research. A successful short transcription therefore demonstrates integration, not the paper's cross-dataset accuracy. Accent, background noise, specialist vocabulary and recording length remain relevant failure conditions.

The system treats transcription as an intermediate result because downstream extraction can faithfully repeat a recognition error. Splitting audio into overlapping chunks addresses

processing constraints, but boundaries can repeat or omit phrases. Silent-input detection avoids asking the model to interpret an empty signal. Duration and size checks are engineering controls; they do not guarantee reliable transcription for every recording below the limits. An independently transcribed test set would be needed to estimate word error rate for the target domain.

Li et al. [10] introduce BLIP with caption generation and filtering to improve image-language pretraining. Their evaluations span image-text retrieval, captioning and visual question answering, with reported gains in recall, CIDEr and VQA score. These are distinct tasks and metrics, not interchangeable evidence of diagram understanding. BLIP was relevant to the early design, but it is not the final deployed captioner. The final pipeline uses OCR, Florence-2 and a fallback captioning model.

Xiao et al. [11] present Florence-2 as a prompt-based model spanning visual tasks, supported by FLD-5B annotations over 126 million images. Its ability to produce task-specific text makes it attractive as an adapter. This project uses a converted base fine-tuned checkpoint for detailed captions, not all tasks evaluated in the paper. A general description can notice objects yet miss a diagram's arrow direction, units or causal relationship. OCR is therefore retained alongside captions, and neither is treated as direct evidence that the underlying visual was understood correctly.

Speech and vision share a design implication: errors introduced before retrieval cannot be repaired merely by attaching a citation. An output may link perfectly to a wrong caption. Keeping origins such as OCR, vision and Whisper in the data model makes that uncertainty visible. It also lets me distinguish raw-file processing from pasted transcripts or descriptions; success with the latter does not demonstrate the former.

2.6 Learning activities and interface design

Roediger and Karpicke [12] compare repeated study with retrieval practice using prose passages. Their experiments test retention after five minutes, two days and one week. Repeated study performs better immediately, while prior testing produces better delayed retention. The contrast shows why satisfaction or immediate fluency is an inadequate learning measure. It motivates practice in this application, but the published free-recall procedure differs from automatically generated multiple-choice questions with feedback. Its effect cannot be assumed for this product.

Mayer's multimedia-learning account [13] emphasizes how verbal and visual material is organized, rather than treating additional media as automatically helpful. This supports separating a dense revision pack into task-focused views and placing evidence near the action that needs it. It does not justify decorative animation or imply that a concept graph teaches a concept. The graph is an exploratory representation of relationships inferred from shared material; a learner may find the underlying passages more useful than the visualization.

WCAG 2.2 [14] provides testable accessibility criteria concerning keyboard operation, visible focus, contrast and alternatives to non-text content. It is a normative reference rather than a usability experiment. The project implements keyboard controls, labels, responsive layouts and reduced-motion handling, but selected browser checks are not a conformance audit. In

particular, a visually polished custom dropdown still needs behavioral testing: the final release exposed a state-mapping defect even though its selected label looked correct.

2.7 Synthesis and contribution

The reviewed work supports a pipeline with explicit boundaries between extraction, selection, generation and review. It also explains why one attractive aggregate score would be misleading. Speech accuracy, source attribution, question usefulness, accessibility and learning retention require different measurements. Bender et al. [15] further motivate scrutiny of fluent model output and training-data risks; their critical analysis is a reason to expose limitations, not a measured error rate for this particular system.

The evaluation traditions differ in ways that affect experimental design. Retrieval research normally fixes a collection and relevance judgments, then compares ranked results. An extraction system additionally needs reference transcripts or visual descriptions. A learning experiment needs participants, controlled exposure and an outcome measured after an appropriate interval. Testing only the final screen collapses these layers and makes failure attribution difficult: a wrong answer could originate in recognition, retrieval, generation or presentation.

For this project, modular evaluation is therefore a methodological choice as well as a software choice. A unit test can establish that a corrupted recording produces a warning, while a reference-transcript comparison would establish something different about words recognized from a valid recording. Likewise, a correct-answer membership check can establish that a quiz is internally usable without judging whether its distractors are convincing. I use the first kind of evidence where it exists and do not substitute it for the second.

This synthesis also changes how model alternatives should be selected. The best candidate is not necessarily the largest checkpoint or the one with the strongest unrelated benchmark. Download size, first-use delay, output structure, language support and recoverability affect whether a model fits the application. The current choices are supported by compatibility and working integrations, but the absence of a matched accuracy-and-resource comparison limits claims that they are optimal. Keeping adapters replaceable is a practical response to that uncertainty.

The project does not claim that no comparable study tools exist, or that combining pretrained models is itself a new scientific discovery. Its contribution is the implemented combination of bounded browser processing, failure recovery, source-balanced selection and an evidence-linked practice workflow. These choices can be inspected and tested separately. The strongest evaluation will therefore show which properties hold, where complexity provides a concrete benefit, and which claims remain outside the evidence.

3 Design (1,843/2,000 words)

3.1 Requirements and evidence

The design starts from a learner task rather than a list of model capabilities: bring material together, understand selected passages, attempt questions and inspect the source when uncertain. Different users may enter this sequence at different points. A learner with a transcript does not need speech recognition; a learner with an image description should not need a vision download. Pasted text remains a first-class input so that expensive adapters are optional rather than prerequisites.

Table 2 maps requirements to their rationale and verification. The rationales have different status. The need for inspectable evidence follows from the attribution problem discussed in Chapter 2. Drag-and-drop and some presentation refinements arose from informal feedback. Performance limits and cancellation came from engineering risk analysis. None of these sources amounts to a statistically representative requirements survey.

Table 2. Requirements and verification routes.

ID	Requirement	Rationale	Verification
R1	Combine notes, transcripts and images	Mixed study materials and template goal	Adapter and mixed-source tests
R2	Keep every study item linked to a source sentence	Learners must inspect answers	Integrity audit and evidence navigation
R3	Preserve valid inputs after recoverable failures	One damaged file should not waste a session	Corrupt-file regression cases
R4	Keep processing stoppable and bounded	Models can exhaust time or memory	Cancellation, timeout and limit tests
R5	Offer local operation without an account	Reduce setup and unnecessary disclosure	Default-engine workflow and code inspection
R6	Support reading and active practice	Study task and retrieval-practice rationale	Practice, retry and flashcard checks
R7	Support keyboard and narrow layouts	Different access needs and devices	Keyboard and viewport checks
R8	Expose comparison and export	Reproducibility and learner control	Benchmark, JSON and Markdown checks

A requirement can be met technically without its intended benefit being established. For example, R6 is satisfied by functioning practice controls, but whether those controls improve recall remains an empirical question. Maintaining that distinction prevents the requirements table from becoming a disguised claim of educational success.

3.2 Architecture and deployment

Figure 1 shows the implemented architecture. Vite produces static assets served by GitHub Pages. The application has no project-operated backend, server database or account system. The main thread owns input controls, display state, native audio decoding and worker lifecycle. A disposable module worker coordinates adapters, preprocessing, retrieval and pack construction. This separation keeps model inference away from most interface interactions and gives cancellation a concrete mechanism: terminating the worker.

BROWSER / USER DEVICE

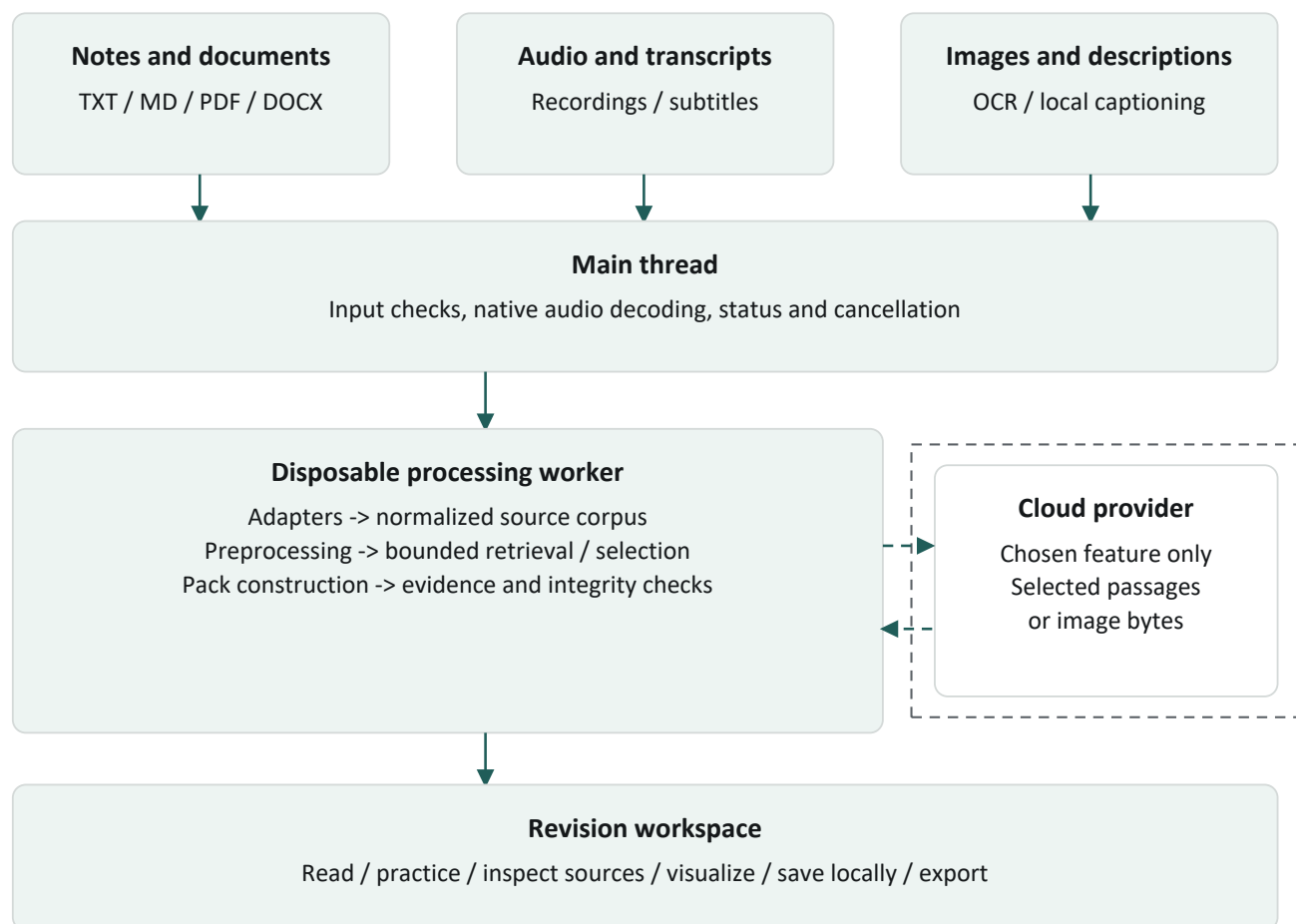


Figure 1. Implemented processing architecture. The dashed external boundary is crossed only for a chosen cloud feature; local model downloads are separate from uploading study content.

The browser-only design resolves an uncertainty in the preliminary report, which proposed unspecified frontend and backend technologies. It simplifies deployment and keeps ordinary study content on the device. The trade-off is substantial: browser memory, storage quotas, codecs and model-download availability become operational constraints. A server could centralize compute and enforce credentials differently, but would introduce hosting, authentication and data-retention responsibilities. For this project, the local-first design is a deliberate scope decision, not an assumption that browsers eliminate security concerns.

Transformers.js supplies model execution through ONNX Runtime [16]. PDF.js and Mammoth handle PDF and Word extraction, while Tesseract.js supplies OCR [18,21,22]. Shoelace components support selected interface elements [17]. These libraries are implementation dependencies, not original research contributions. The project's own code supplies the orchestration contracts, source records, retrieval combination, study-pack structure and interaction logic that connect them.

3.3 Data representation and provenance

The source store is an in-memory corpus, not a remote database. An adapter returns a source with a label, normalized text, origin, adapter type, optional warning and structured blocks. A block may retain its heading, kind and PDF page. Preprocessing creates sentence records with an identifier, source label, sentence index, text, content tokens, section, page and origin. Figure 2 shows how these records connect to generated items.

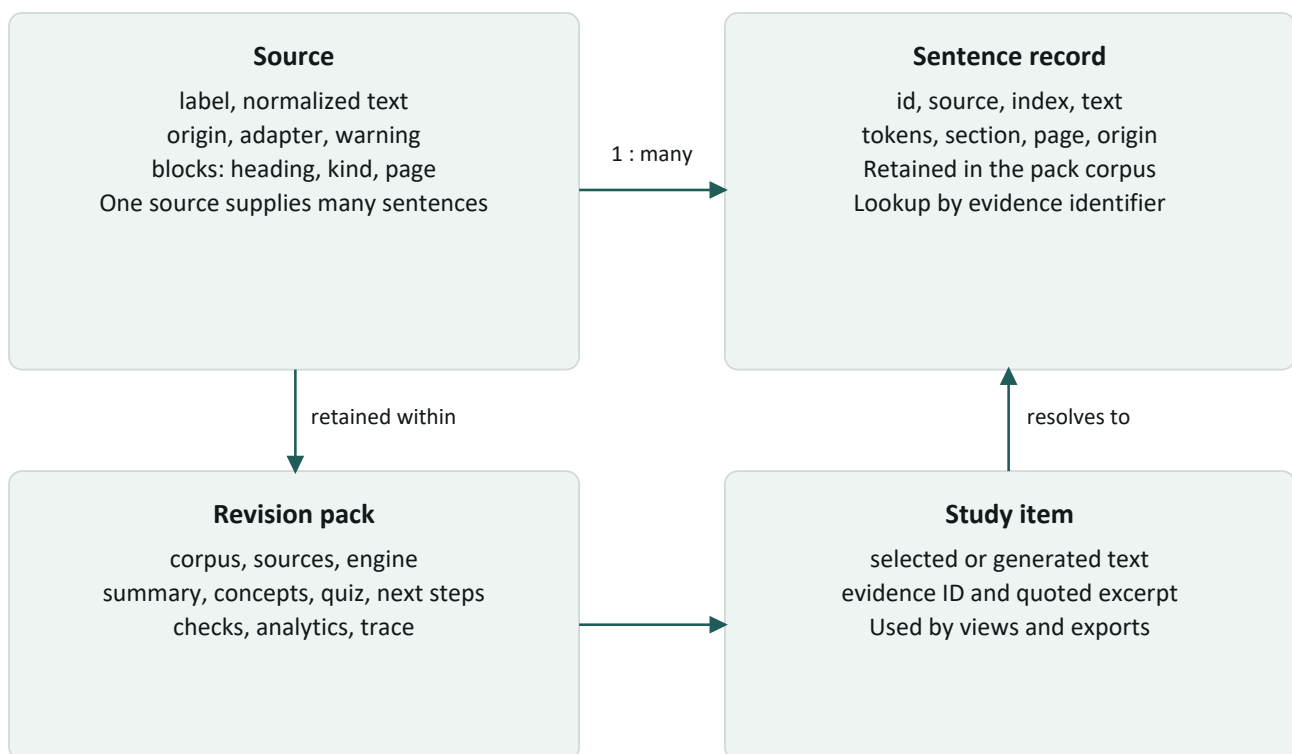


Figure 2. Source and revision-pack relationships. Evidence identifiers connect outputs to retained sentences; they do not establish that a sentence is factually correct.

The identifier combines a disambiguated source label with its sentence index. Duplicate labels are handled during collection. This is sufficient for stable references within a generated pack; it is not a globally persistent citation scheme across edited inputs. Regenerating the pack creates a new corpus. The source index can retain gaps after sampling, so displayed sentence numbers need not be consecutive.

A revision pack contains the corpus, source metadata, engine identity, summary, concepts, quiz, next steps, checks, analytics and an orchestration trace. Summary and quiz records carry an evidence identifier and excerpt. The same structure feeds reading, practice, source search and export, avoiding a second independently reconstructed evidence store in the interface. That shared representation is important because a display-only source label could otherwise become detached from the passage it claims to represent.

Local history retains up to eight packs and validates their shape before loading. Practice state is associated with the pack so that reopening one session does not silently overwrite another. Storage failure leaves the current in-memory result usable and reports that saving failed. JSON and Markdown export provide an additional escape route, but browser storage is not a backup service and raw attachments are not a portable archive of the original files.

3.4 Control flow and failure policy

The worker request contract uses an action, inputs and engine; preview and explanation-polishing actions have narrower payloads. Messages back to the main thread distinguish status, stage, completion and error. Figure 3 shows the successful path and the cancellation boundary. The interface only replaces the current pack after a successful result, which protects the previous work when a later generation fails.

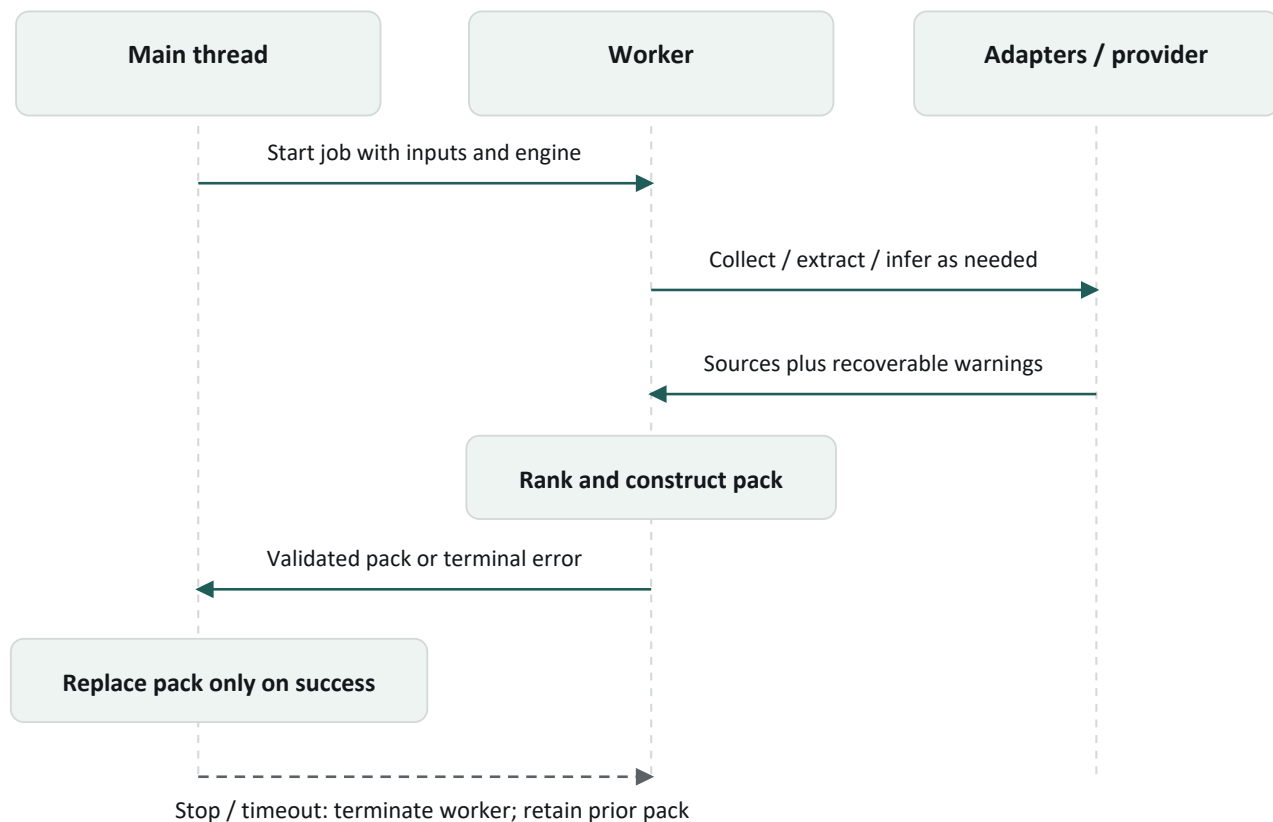


Figure 3. Generation lifecycle. Recoverable adapter warnings travel with the successful pack; cancellation and hard safety limits stop the job without replacing the previous result.

Failures are classified by whether continuing would be meaningful and safe. A corrupt document or unusable recording can produce a named warning while valid pasted material continues. Empty input, an exceeded safety limit or cancellation stops the job. A malformed cloud answer is rejected because partially displaying it could create broken citations or invalid answer choices. This policy makes failure handling part of the data contract rather than an incidental alert.

Input limits are explicit: 16 attachments, 256 MiB combined, 25 MiB per document, 10 MiB per image, 100 MiB per recording and 30 minutes of recordings per run. Pasted and extracted text are bounded at 600,000 characters, PDFs at 300 pages and the retained corpus at 1,400 sentences. The interface labels byte limits as MB, although the constants use binary multiples. These are safeguards rather than guarantees that every supported device can handle the maximum workload.

3.5 Interaction design

The landing page offers an example and three input routes. After generation, six destinations separate Overview, Read, Practice, Explore, Sources and Edit inputs. Figure 4 shows the entry interface. The separation reduces the need to scroll through every output type when the learner has one immediate task. Editing does not immediately destroy the existing pack; the interface explains when a new generation will start a separate practice record.

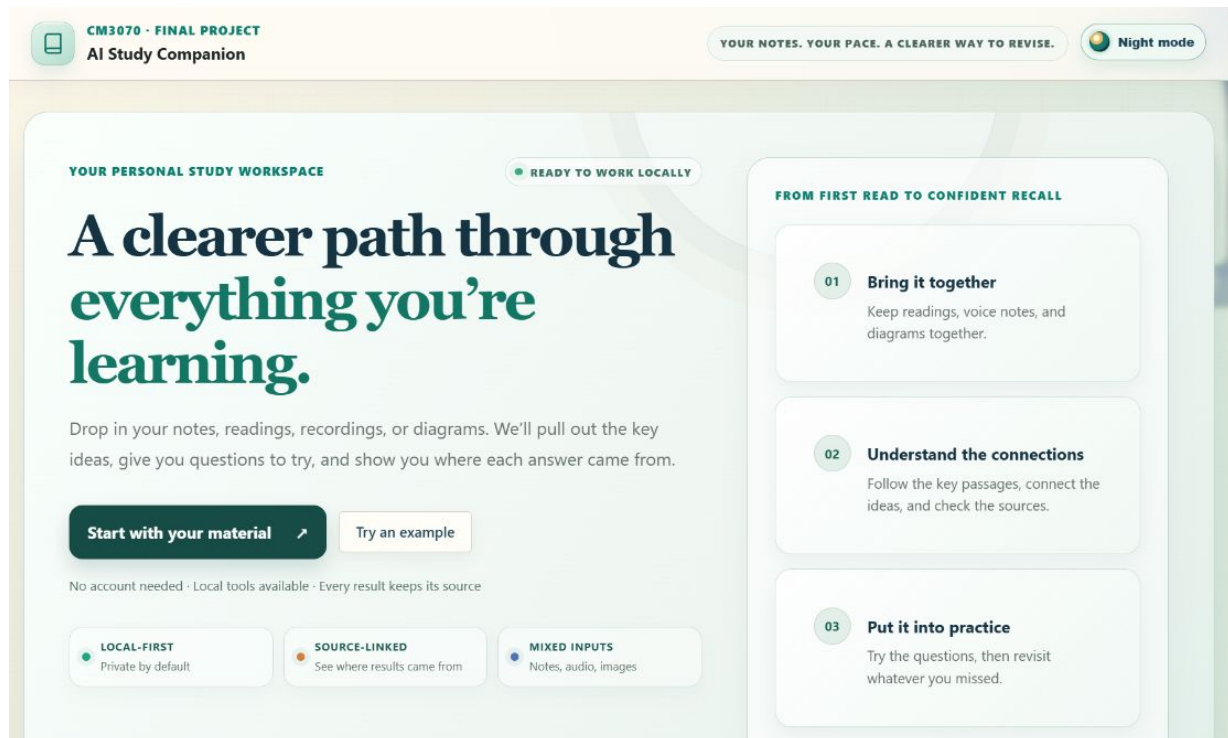


Figure 4. Current application entry screen using the public release. The example and material-entry route make the first action explicit.

Reading keeps evidence links near the selected passages. Practice initially hides answers, records first-attempt correctness and offers source review after an attempt. Sources supports textual search plus source and citation filters, making unused material accessible. Explore presents source coverage, grounding distributions, quiz types and concept connections. These are inspection tools, not independent measurements of comprehension.

Daylight and night themes use a shared token system. The fluid cursor effect is third-party decoration [19]; it is not part of the retrieval or study mechanism. Reduced-motion handling and hiding the layer during processing reduce visual interference, but hiding a canvas does not prove that its underlying animation loop releases GPU resources. This remains a performance limitation. Custom controls also require their own keyboard and state tests; replacing native appearance does not automatically improve accessibility.

3.6 Privacy and trust boundaries

There is no login because the application does not operate a multi-user service. That reduces account-management complexity but does not authenticate someone using the same browser profile. Stored packs and the optional API key use localStorage. They are not encrypted application secrets and are accessible to scripts running in the same origin. I therefore do not describe the tool as suitable for confidential material merely because it is local-first.

Cloud processing is an explicit choice. The user supplies a Google key, selects the cloud engine and acknowledges a free-tier project with billing disabled. The application cannot independently verify billing status. Selected passages, quiz excerpts or image bytes are sent according to the chosen feature; the key is omitted from pack export. Local models still require initial network downloads, so local processing is not equivalent to guaranteed offline availability from a fresh browser.

Uploaded text and model responses are treated as untrusted content. Rendering escapes text, prompts instruct models not to follow embedded source instructions, and responses undergo structural validation. These measures address concrete risks but do not constitute a complete security audit or a solution to prompt injection. The safest intended materials are non-sensitive study examples. Formal research involving users would additionally require the course's consent and ethics process; informal feedback is not presented as evidence of such approval.

3.7 Alternatives and development decisions

Two learner scenarios help test the coherence of these choices. In the first, a student pastes short notes and a transcript and wants a quick first pass. Hybrid Focus requires no generative model or API key, and the same pack can be read, practiced and exported. In the second, the student adds a recording and diagram. The heavier adapters normalize their outputs before the same pipeline runs. If the diagram fails, its warning remains visible and successful notes can still produce a pack. The second path adds uncertain extraction rather than pretending that every input format is equally reliable.

A chatbot-first interface is an alternative to the chosen interaction. A free-form conversation would require maintaining dialogue state and repeatedly associating answers with sources. A structured pack gives the application a smaller, testable output contract and lets the learner revisit a fixed set of material. The trade-off is reduced flexibility: learners cannot ask arbitrary follow-up questions in the local engines, and predefined exercises may not match every discipline.

I worked on the project from May to September, beginning the proposal, concept and previous-work research on 18 May. The feature prototype and preliminary report came before the August draft report; August was not the start of implementation. I built on the prototype's shared workflow by adding real media inference, separated workers, stronger source records and practice state, then fixing interface regressions. My project log follows that work and the tests I carried out along the way. The later entries include the fullest saved browser checks and test totals, but testing was part of the work before September too. I keep planned features separate from capabilities I verified.

4 Implementation (2,242/2,500 words)

4.1 Organization and reproducibility

The implementation separates adapters, runtime control, pipeline logic, evaluation, exports and interface modules. The principal entry point coordinates form state and processing jobs rather than embedding model-specific behavior in page handlers. The lockfile records dependency versions; the verified build used Vite 6.4.3. The deployment workflow runs tests, builds assets, executes the offline benchmark and publishes the successful build to GitHub Pages. This makes the deployed revision identifiable, although external model files and cloud services can still change independently.

The discussion below refers to the source files in the public repository [20]. Naming the functions makes the implementation easier to inspect: the technical contribution lies in how they work together, not in the length of the dependency list. Any reader can follow an algorithm into its tests and compare the description with the code.

4.2 Ingestion and model adapters

Text collection supports pasted notes and multiple TXT, Markdown, PDF and DOCX files. PDF.js extracts positioned text. The PDF adapter detects a repeated central gutter and reads clear two-column bands down each column, keeping spanning headings separate. Ambiguous layouts retain row ordering. Document cleanup reconstructs blocks and removes recurring page furniture. Mammoth extracts Word text before cleanup. Headings and bullets help later section allocation. A scanned PDF without a text layer still needs another input route; image OCR does not automatically make scanned PDFs readable.

The audio adapter distinguishes recordings from transcript files, including subtitle formats. Subtitle handling removes cue numbers and timestamps without stripping meaningful numeric lines from ordinary transcripts. Native audio decoding occurs before transfer to the worker; decoded samples are resampled for inference. The PCM buffer is transferred rather than copied, avoiding a second large buffer at the worker boundary. Whisper base.en uses quantized q8 weights on WebAssembly and 25-second chunks with four-second overlap. The transcriber is disposed after a recording.

The final recovery change handles metadata and decoding failures before they can abort otherwise valid mixed input. A failed file becomes a plain descriptor containing its name, type, size and preparation error. The worker adapter turns that descriptor into the established warning path. Cancellation remains distinguishable from a damaged file, and duration-limit checks remain hard stops. This avoids the misleading result of accepting a cancelled job as merely another recoverable warning.

Image processing combines English OCR with detailed local captioning. Florence-2-base-ft runs on WebAssembly with q8 weights; the detailed-caption task uses a maximum of 160 new tokens, three beams and repeated-ngram suppression. A ViT-GPT2 captioner is available as a fallback. OCR cleanup uses recognized-word confidence and layout information before merging readable text with the description. Caption cleanup rejects

unusable outputs, but it is not a semantic verifier. A plausible incorrect description can survive these checks and requires human review.

The adapters retain distinct origins so the user can tell a pasted description from OCR or model-produced text. They return warnings separately from successful sources. This is significant for mixed files: dropping one unreadable image should not remove valid lecture notes, while a run with no usable material should not return an apparently successful empty pack. Tests exercise both the successful collection contract and simulated model failures.

4.3 Preprocessing and source balance

In `preprocessSources`, each source is converted into sentence records before applying the 1,400-record cap. Quotas are allocated round-robin across nonempty source pools until the budget is consumed. Within a long source, retained indices are spread from the beginning to the end rather than taking only the prefix. Selected records are then interleaved across sources. This protects a short transcript from being crowded out by an earlier, much longer document.

The approach has a cost: evenly spaced sampling can omit the very paragraph a learner needs. The pack therefore records truncation and warns that passages were sampled. Coverage calculations use the retained corpus, not every sentence in the original files. This denominator matters. Reporting complete coverage of retained records would not mean the whole original reading had been represented.

Section records retain source and heading relationships. Summary construction can allocate space across sections for a single structured document or reserve source representatives for mixed input. Exact normalized duplicate sentences are excluded from additional selections. The initial representative choices are still constrained by usable candidates and available summary space; the mechanism is a safeguard against domination, not a guarantee that every source deserves equal emphasis.

4.4 Retrieval and summary construction

Keyword Scout supplies the comparison baseline. It extracts frequent terms and adjacent phrases, selects matching evidence and constructs simple question templates. Its summary scoring considers keyword hits, sentence length and source origin. It also received the common source-balancing fixes, so it is not a frozen copy of the early prototype. Comparisons in Chapter 5 are between the current engines under shared safeguards.

Hybrid Focus builds TF-IDF sentence vectors and a BM25 index. In `bm25.js`, k_1 is 1.5 and b is 0.75. The implementation uses inverse frequency $\log(1 + (N - df + 0.5)/(df + 0.5))$, term-frequency saturation and length normalization. TF-IDF provides a second lexical view. Rankings are fused in `rrf.js` by adding $1/(60 + \text{rank})$ for each ranking list, then normalizing the fused scores [4,5]. These parameters are explicit implementation choices, not experimentally optimized educational settings.

The hybrid provider first selects at most 72 candidates and up to 12 anchor sentences. Sentence-to-anchor similarity supplies another ranking signal. In the default mode this uses sparse TF-IDF similarity; Semantic Explorer substitutes MiniLM sentence vectors. It does

not encode every query into a new dense vector during this ranking path. Candidate and anchor caps bound repeated similarity comparisons, while cached query scores avoid recomputing identical term lists.

Summary candidates combine lexical hits, length and origin factors, retrieval relevance and a study-value heuristic that downranks administrative material. The selector uses MMR with λ 0.72: weighted relevance is reduced by weighted maximum similarity to already selected material [6]. Redundancy maxima are updated incrementally after each choice. This avoids repeatedly comparing every candidate against the entire selected set at every round. The selected source sentences are then restored to corpus order for presentation.

These heuristics are interpretable but imperfect. The relevance term is not a calibrated probability, and its scale is not identical to the similarity penalty. The High, Medium and Low source-relevance labels derive from thresholds on this score, not measured confidence intervals. Extractive summaries preserve the wording of a source sentence but can omit qualifications in neighboring sentences. Those limitations are reflected in the source-review workflow rather than hidden behind the term grounding.

An important computational distinction is between model inference and ranking. Building an embedding for every retained sentence can remain expensive even though the later candidate-to-anchor comparisons are capped. The cap limits one stage, not the whole algorithm. With n retained records, at most 72 candidates and 12 anchors, each query's bounded similarity stage performs at most 864 comparisons, while lexical scoring still examines the corpus. Score caching helps repeated queries but does not remove the first pass or its memory cost. The 1,400-record budget therefore complements rather than duplicates the ranking cap.

The summary code illustrates the separation of policy and mechanism. Candidate scoring determines which sentences appear useful; source or section allocation determines which groups receive space; MMR decides how to spend the remaining space without excessive redundancy. These choices can be changed independently because they operate on explicit records rather than strings embedded in interface markup. Tests can supply a replacement retrieval provider and observe whether evidence selection changes, which is stronger evidence of integration than checking that a model was imported.

4.5 Cloud generation and validation

Study Reasoner first constructs the local corpus and concepts, then asks the configured Gemini endpoint to write summary and quiz items from selected evidence. The implementation's model identifier is gemini-3.8-flash. The request uses the generateContent API with a JSON response schema, temperature 0.2, an output-token budget and an API-key header. Evidence selection is bounded to 100 passages and 45,000 sentence characters. This describes the integrated configuration; future availability of that provider identifier is not guaranteed.

validateStudyResponse requires nonempty summary and quiz arrays. Each item must cite a supplied identifier and contain a quote found in the normalized source sentence. Questions need two to four distinct, nonempty choices, and the answer must exactly equal one choice.

Truncated, malformed or invalid responses fail without replacing the previous pack. This contract is more useful than accepting arbitrary model prose and trying to infer its citations afterward.

The Google client retries HTTP 500 and 503 twice with the same model, within a two-minute request deadline. Authentication, quota and availability errors have distinct messages. There is no automatic switch to a paid model. A valid quote still does not prove that a question has only one defensible answer or that a generated explanation follows from it. The validation checks format and traceability, leaving semantic correctness as a separate evaluation problem.

4.6 Questions and persistent practice

The local quiz generator favors explicit definitions and causal clauses before using cloze and concept-matching templates. It only constructs a causal question when the source includes a suitable because clause; graph edges are not treated as causal evidence. Distractors must differ from the answer, and options are shuffled while retaining the correct answer text. Live sessions use Math.random; seeded randomness is reserved for reproducible tests and benchmarks.

Question selection limits repeated evidence. Subsequent practice rounds prefer unused passages and use fingerprints to reduce repeated items. When insufficient material remains, the interface explains that evidence may be reused instead of implying an endless supply of new questions. The saved pack is not silently rewritten by a follow-up deck. Figure 5 illustrates the learner's answer and optional confidence controls.

QUESTION 1 / 3 0 correct on first attempt

Concept - Text notes

Which idea does this passage describe? “Retrieval augmented generation is a technique that combines a language model with an external source of information.”

How sure are you?

Unsure Fairly sure Sure

1 Language Model

2 Retrieval Augmented

3 Revision Pack

4 External Source

Figure 5. A generated practice question. The answer choices, confidence prompt and source-based wording are visible; the screenshot is an interface example, not participant data.

Practice state records first attempts separately from later answers. Retry-missed and weaker-source paths therefore respond to actual session events rather than a single aggregate score. Flashcards provide reveal, rating and navigation states with in-session Leitner-style boxes. Optional confidence selections map to numeric values and support a Brier-style squared-error measure against correctness. These features provide reflection prompts; the assigned probabilities, readiness score and review schedule have not been calibrated against learner outcomes.

4.7 Traceability, search and visualization

integrity.js checks whether output identifiers resolve, evidence quotes match, output labels repeat, replacement characters occur, and quiz answers belong to valid choices. It reports structural issues rather than repairing factual claims. Coverage separately counts which retained sentences and sources are cited. grounding.js computes overlap and a heuristic score: complete claim-token overlap returns 100; otherwise the rounded score combines token F1 with retrieval relevance at weights 0.7 and 0.3. The implementation's recall denominator is the claim-token set, so I treat the displayed measure as lexical overlap rather than standard retrieval recall.

The Sources view uses a BM25-backed search function with query, source and citation fields. Its recent defect was a mismatch between those state keys and the visual control identifiers. Selecting an option changed the displayed label but wrote deskSource or deskCitation instead

of source or citation. Explicit mapping fixed the behavior. Figure 6 shows the corrected Audio transcript and Not cited yet combination returning two sentences from the built-in example.

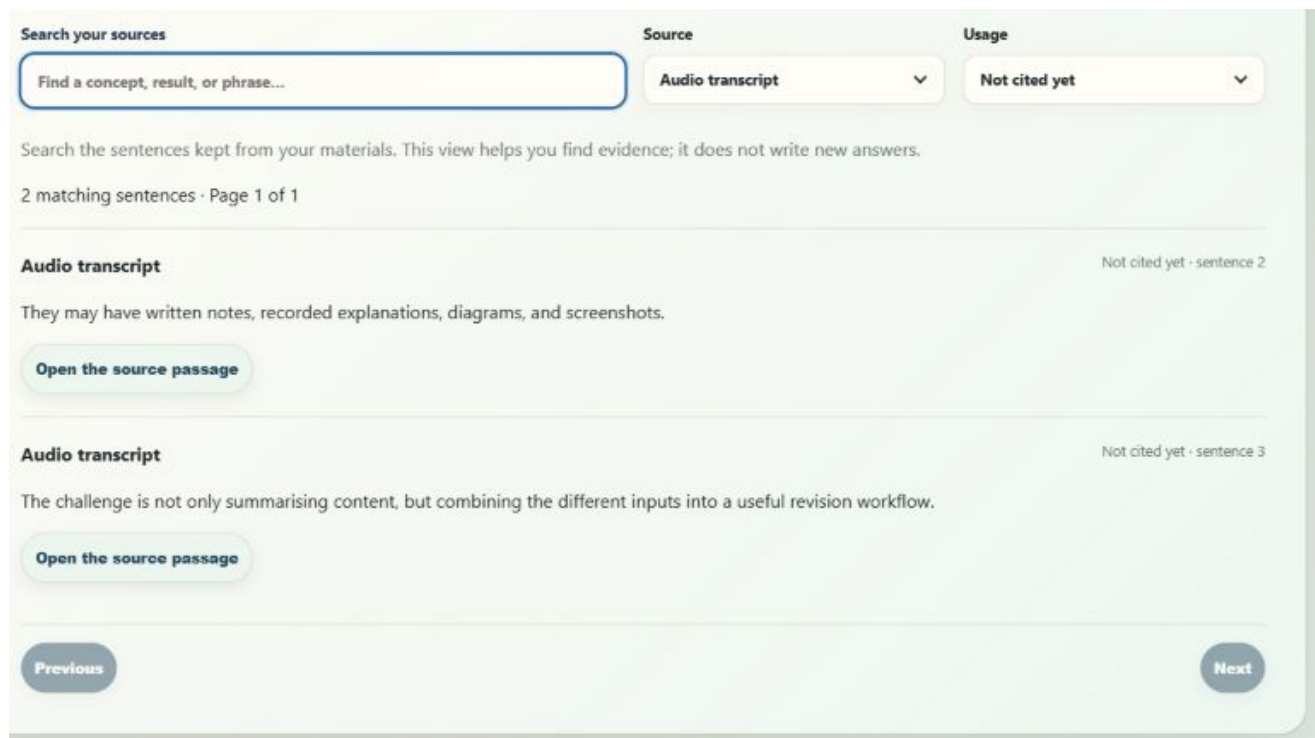


Figure 6. Corrected source filtering on the public release. The selected source and citation state jointly restrict the displayed passages.

Concept edges combine shared terms, shared evidence, section and source information. A bounded PageRank-style iteration supplies a connectivity signal; isolated-node handling means it should not be interpreted as a normalized probability distribution.

Two-dimensional SVG and three-dimensional canvas views offer exploration and evidence navigation. Their relationships indicate textual association, not verified conceptual dependence. Counts, percentages, histogram bands and graph strength controls change the presentation of the same pack.

4.8 Interface engineering and release controls

Shoelace components coexist with application-specific controls and semantic HTML. Theme work therefore required styling component parts as well as ordinary CSS. Earlier pale badge borders and nested-looking toggles illustrated why a library alone does not ensure a coherent result. The final dropdowns use custom listboxes with keyboard selection, Escape handling, focus state and preserved values; behavioral tests now verify results as well as labels.

Vitest covers algorithmic and adapter behavior, with jsdom integration tests for interface state. Worker tests exercise completion, crash, timeout, transfer errors and cancellation. The repository includes comparison helpers, a consolidated evaluation log and sample inputs. These include original NASA research and media, labelled format conversions, and

deliberately empty, corrupt or oversized files. Here, a test fixture means a repeatable input prepared for a check; it does not imply a new research dataset or a real user observation. Passing these checks does not measure line coverage or eliminate integration risk, but it provides repeatable protection for the invariants most central to the project.

Release reproducibility depends on more than rerunning the development server. The production site uses a repository subpath, separate worker assets and dynamically loaded parser/model bundles. A successful local interaction can therefore coexist with a deployment-path failure. The CI build and a post-deployment sample check cover different parts of that risk. Model requests remain an external dependency, and an exported pack is useful when a subsequent service or download becomes unavailable. I date the application checks without claiming that the complete external environment is permanently frozen.

5 Evaluation (2,414/2,500 words)

5.1 Questions and method

Evaluation follows the four objectives rather than using a single overall quality score. For O1, the question is whether supported inputs can reach a usable corpus and whether recoverable failures preserve other material. For O2, it is whether output links and structures are valid. For O3, it is whether the interface supports the intended tasks without losing state. For O4, it is whether the more selective engine changes the resulting pack in a defensible way. Learning effectiveness is outside the completed evaluation.

I use four kinds of evidence: automated regression tests, a reproducible synthetic comparison, browser interaction checks and informal feedback. Model smoke tests in the evaluation log provide additional integration evidence. The distinction matters because a mocked failure, a real-file import and a live model run establish different things. I tested throughout development; the most detailed saved browser and integration checks are from 12-22 September 2026. I ran the offline comparison on 21 September and followed it with a repository and sample-library check on 22 September. Those dates do not mark the start of evaluation.

The release criterion for structural tests was no unresolved evidence identifiers, invalid answer choices or failing assertions in the specified cases. Mixed-input recovery required retaining valid sources and naming the failed file. Interface filtering required the returned records to change correctly, not merely the selected label. These criteria are measurable, but they are engineering acceptance conditions rather than a validated output-quality scale.

5.2 Automated and browser results

The 21 September release check passed 228 tests across 51 files. After the sample-library review and PDF reading-order correction on 22 September, the suite passed 245 tests across 53 files, and the production build completed. The suite includes retrieval, sentence processing, grounding calculations, practice state, export, storage failure, document parsing, worker lifecycle and input limits. It also includes adversarial or awkward cases such as Unicode, negation, duplicate labels, escaped markup and corrupt Word input. These results provide repeatability across many boundaries; they are not a percentage of all application behavior or proof that every branch is covered.

Table 3 summarizes the evidence most closely tied to the objectives. The browser results include checks completed during the release pass and a fresh sample generation and filter check during report preparation. No application warning or error was observed in that fresh production-console check. Absence of a console error is useful operational evidence, but an incorrect result can still be produced without an exception, as the filter bug demonstrated.

Table 3. Engineering verification and its scope.

Area	Observed result	What it does not establish
Regression suite	245 tests passed in 53 files	Exhaustive branch or device coverage
Production build	Static bundle generated successfully	Runtime correctness by itself
Source filters	All 12 sample combinations verified; exact-record regression added	Search relevance on arbitrary material
Mixed files	Valid Word/PDF material retained alongside corrupt files	All document layouts or codecs supported
Audio recovery	Invalid metadata and decoding errors follow warning path	Speech-recognition accuracy
Model failures	Mocked image/audio failures retain pasted material	Real inference on those failed examples
Workspace	Six views, practice, history, exports and charts exercised	Independent usability or accessibility certification
Narrow layout	390-pixel layout checked without horizontal document overflow	Physical-device or assistive-technology coverage

For the built-in sample, all sources produced 11 searchable sentences: four cited and seven uncited. Notes contributed five, split two cited and three uncited; the transcript and image-description lanes each contributed three, split one and two. Selecting a source after visiting a later page reset pagination. Keyboard selection and leaving and returning to the view preserved the intended filter state. These counts provide concrete oracles for the corrected interaction rather than relying on a screenshot alone.

The mixed-file tests are particularly important for O1. Real PDF and DOCX files were paired with damaged files in the browser. Generation retained usable material and displayed the failed filenames. Automated tests additionally covered metadata and decoding failures and four mixed-input combinations: corrupt document, audio, image and all three. The image-model failure was simulated at the adapter boundary. That scope supports the recovery contract without pretending to measure visual recognition.

Replacing the sample library with real space and AI material exposed another input-quality issue. The NASA research paper had two columns, but the PDF adapter initially joined fragments across each horizontal row. The resulting text mixed unrelated sentences. The adapter now checks for repeated central gaps between substantial text runs and reads clear

two-column regions down each column, keeping spanning headings between regions. Tests cover the actual paper, single-column pages, spanning headings, short table cells and missing width data. This is a conservative layout heuristic, not a general solution for every journal or magazine design. The sample review also caught an image above the 10 MiB limit; NASA's smaller web rendition replaced it, and a regression check now verifies ordinary samples against per-file limits.

5.3 Offline engine comparison

The benchmark uses five authored synthetic cases: four biological definitions; seven sentences distributed across reading, transcript and diagram-description sources; four sentences containing numbers and negation; 100 repetitive reading sentences plus one privacy-related transcript sentence; and a single sentence. These cases probe particular risks rather than approximate a representative student corpus. Both engines receive identical sources, with three repetitions and alternating execution order. Seeds 42, 43 and 44 make option ordering reproducible, giving $5 \times 2 \times 3 = 30$ runs.

The benchmark ran under Node 24.13.1 on Windows x64 with an Intel Core Ultra 9 285K. It measures the offline pipeline after source collection, excluding file decoding, model downloads, inference and rendering. All runs completed with no structural-integrity issues, 100% valid output links and 100% expected-term recall. Expected-term recall is a substring check across summaries and concept labels against a short authored term list. It does not test semantic coverage, correct negation or factual accuracy.

This ceiling is itself a finding. On these small cases, the link and term metrics cannot distinguish the engines. A claim that Hybrid Focus is more accurate because it passes them would be unsupported. Table 4 instead compares the number of questions with the number of distinct evidence sentences behind them for the seed-42 outputs. Repetitions change ordering but do not turn the five cases into 15 independent datasets.

Table 4. Questions and distinct supporting sentences in the offline comparison. Each cell shows questions / distinct evidence sentences.

Case	Keyword Scout	Hybrid Focus	Summary sources represented
Definitions	5 / 1	4 / 4	1 / 1 for both
Mixed sources	5 / 2	4 / 4	3 / 3 for both
Negation and numbers	5 / 2	4 / 4	1 / 1 for both
Imbalanced inputs	5 / 1	2 / 2	2 / 2 for both
Single sentence	5 / 1	1 / 1	1 / 1 for both

Figure 7 makes the trade-off visible. The baseline produces 25 questions supported by seven distinct within-case evidence selections, whereas Hybrid Focus produces 15 supported by 15. This reduction reflects the newer engine's unique-evidence policy, not an independently measured learning benefit. A question can use different evidence and still be poor; conversely, several well-designed questions can legitimately examine one passage. The result supports reduced repetition in these cases, not a universal rule that one question per sentence is optimal.

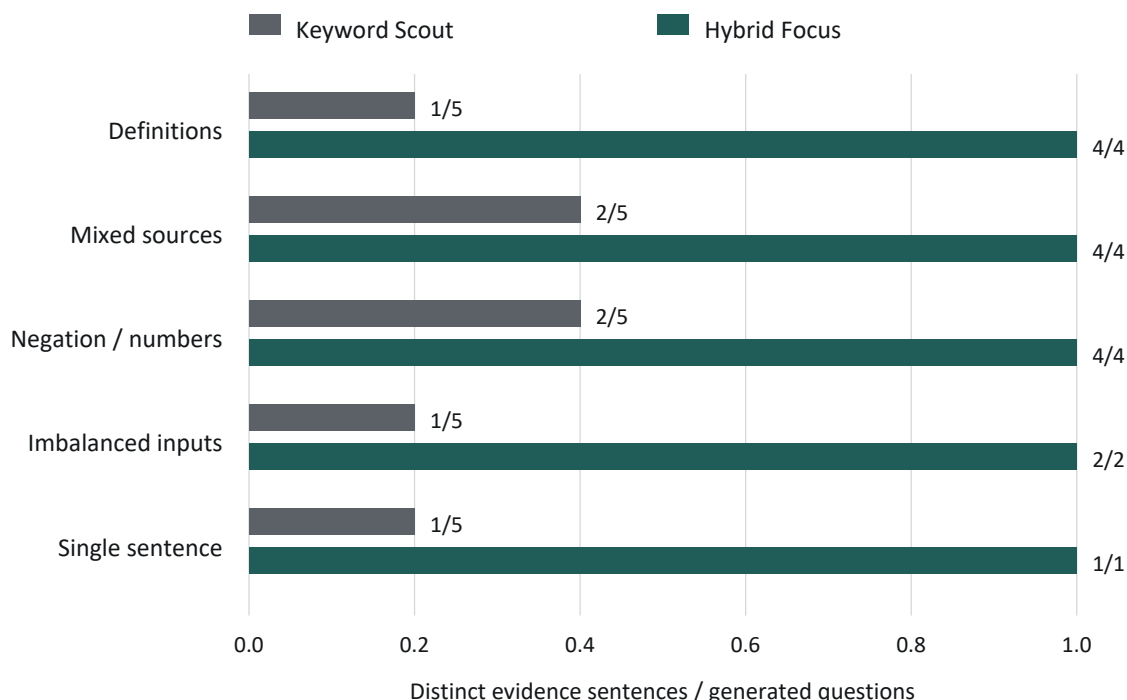


Figure 7. Distinct supporting sentences per generated question for the five synthetic cases. Higher values indicate less evidence reuse in this comparison, not greater factual accuracy or learning effectiveness.

The single-sentence case illustrates the qualitative difference. Keyword Scout creates cloze answers including Checking Helps, Helps Students and Students Review. These are legal phrase matches but weak study concepts. Hybrid Focus produces one concept-matching question for Source Checking. This is a concrete improvement in restraint, although the correct phrase is visible in the quoted passage, so the item may test recognition more than unaided recall. The implementation has reduced one failure mode without solving pedagogical question quality.

Source balance also shows why the comparison needs care. Both engines represent all available sources in their summaries, including the short privacy transcript beside a long repetitive reading. They share the balancing safeguard, so that result cannot be attributed uniquely to hybrid retrieval. The imbalanced case produces six summary sentences under

Hybrid Focus and four under the baseline, but this evaluation does not establish which length a learner would prefer.

Median observed pipeline times by case were 61, 63, 63, 63 and 63 milliseconds for the baseline, and 62, 49, 62, 63 and 62 milliseconds for Hybrid Focus. Across individual runs the ranges were 48-64 and 49-63 milliseconds respectively. These close values do not support a speed ranking. The pipeline retains asynchronous zero-delay yields, and Windows event-loop scheduling contributes to the measured duration. Three repetitions are insufficient for robust latency estimates, especially on a high-specification desktop. No significance test or population confidence interval is reported.

5.4 Model integration and observed failures

Integration testing recorded in the evaluation log went beyond the offline benchmark. An MP3 transcription initially failed to fetch its model, then succeeded on retry. A local sample image completed OCR and detailed description; stopping an image run preserved the attachment. Semantic Explorer completed a model-backed retrieval run. A separate six-minute repeated-speech recording completed in about 77 seconds on the development machine. This was synthetic repeated speech, not a varied lecture benchmark. These are observations from earlier development checks, not measurements repeated for every report revision.

The same logs record a 64 MiB WAV-container check whose size came from an ancillary JUNK chunk around short speech. It is evidence of file-size handling, not long-duration performance. They also record successful cloud generation, diagram description and explanation polishing, alongside repeated HTTP 503 overload. These observations establish working integration at those times, while demonstrating a dependency on network and provider availability. They do not supply word error rate, caption accuracy or independently rated cloud-answer correctness.

No controlled benchmark compared alternative speech or vision checkpoints on the same labeled material. That remains a limitation against the template's emphasis on evaluating pretrained-model choices. The final report can identify the actual configurations and observed behavior more precisely than the draft, but it cannot replace missing model-selection experiments with a literature citation. Optional semantic and cloud paths were not comprehensively rerun during this report-preparation check.

5.5 Informal feedback and design changes

My informal feedback came from my mother, father, adult sibling and a software-developer friend during development. I did not recruit them through a controlled study. I later wrote down what I remembered, but I had not kept individual session dates, transcripts, exact session counts, questionnaire scores or measured learning outcomes. I discuss the feedback by the changes it suggested, not as a formal analysis of recorded interviews.

My mother's instinct was to drag files into the application. That exposed a mismatch between her expected action and the original import-only interaction. Her suggestion prompted drag-and-drop, and she later identified a persistent drop overlay that required a refresh. I

reported correcting it and receiving positive feedback on a later use. Current code and tests demonstrate drop handling, but the exact historical build and retest date are not available. The useful lesson is about discoverability and recovery, not a numerical usability improvement.

My father used transcript material to review meeting content through summaries and quizzes. He also noticed strange-looking output and weak or inaccurate questions. This feedback matters because successful generation is not sufficient when the generated exercise is awkward or misleading. Subsequent engineering work addressed text cleanup, evidence selection and question repetition, and the synthetic comparison exposes related weaknesses in the baseline. There is no recorded before-and-after rating from him that would establish the size of an improvement.

My adult sibling used school notes and readings and expressed satisfaction, but no specific measured benefit or detailed improvement request was recorded. The developer friend suggested the fluid cursor and greater contrast and gave positive visual feedback. Neither observation establishes comprehension, accessibility or expert-review validity. Family familiarity and enthusiasm for a student project may bias responses. I therefore treat these accounts as formative design input, not confirmation that the application meets the needs of students generally.

5.6 Critical assessment and validity

O1 is supported by real-file workflows, adapter tests and partial-failure recovery. Its limits are model accuracy, browser codecs and constrained input budgets. O2 is strongly supported for structural traceability, but not semantic entailment. O3 has developer-led interaction evidence and useful informal feedback, but no independent task-completion study. O4 has a reproducible baseline comparison demonstrating reduced evidence reuse; it does not establish overall superiority of hybrid or model-backed generation.

Several threats constrain interpretation. Synthetic fixtures were authored with knowledge of the application and contain small, relatively clean texts. Their term-recall metric has a ceiling and can reward superficial matches. The two engines share safeguards and differ in several generation rules, so this is not an isolated retrieval ablation. Browser checks were concentrated on one Windows environment; emulated width is not a physical phone. Model logs are smoke tests, while many failure tests use mocks. Informal users were a convenience sample with no standardized procedure.

Two repaired defects show how the tests improved. The source-filter regression originally asserted the visible selected option, which could pass while the search state remained unchanged. Its replacement checks exact returned sentences across every source/citation combination and after navigation. The audio regression similarly moved beyond testing errors inside the worker: native metadata and decode failures occurred earlier, outside that recovery path. Tests now cover those boundaries explicitly. The improvement is a better failure model and test oracle, not merely a larger test count.

An earlier quiz defect also made the second option predictably correct because live generation used a constant random value. The corrected implementation uses

nondeterministic choice shuffling for live sessions and reproducible seeds only for evaluation. This matters pedagogically even when every question has a valid citation: a learner could exploit position rather than study the material. The case reinforces the need to test interactions across a sequence of questions, not just the internal validity of one item.

The strongest general lesson is methodological: visual state, structural validity and meaningful output require separate tests. The dropdown bug survived label-only tests; the baseline's weak phrases survived a complete source-link audit. Future evaluation should therefore add human judgments of support, clarity and usefulness without replacing the engineering checks. A consented adult-student study could counterbalance two matched materials and compare manual preparation with the app, measuring time, unsupported claims detected and delayed recall. Independent raters would need an agreed rubric and recorded disagreements. That is future work, not evidence claimed here.

6 Conclusion (410/1,000 words)

6.1 Outcome

AI Study Companion demonstrates a working implementation of the selected orchestration template. It accepts mixed materials, normalizes them into a shared corpus, builds a structured revision pack and connects study outputs to retained source passages. The final application goes beyond the draft's simulated media inputs: local speech recognition, OCR, image description and semantic embeddings are integrated, with an optional cloud generation path. A public static deployment makes the result accessible without installing the development environment.

The most substantial contribution is the relationship between processing and inspection. Source identifiers are carried through extraction, selection, practice, search and export. A learner can inspect both used and unused retained passages. Bounded processing and partial-failure recovery make the workflow usable when a file or model stage fails. These are concrete software-engineering contributions built around existing research and pretrained models; they are not claims of a new foundation model or a novel theory of learning.

6.2 What the evidence supports

The final regression run passed 255 tests across 54 files, and the production build succeeded. Thirty synthetic comparison runs completed without structural-integrity failures. The direct engine comparison showed a useful difference that aggregate grounding scores missed: Hybrid Focus generated fewer questions with less reuse of the same evidence, while the baseline sometimes produced weak adjacent-word cloze answers. Both engines preserved source representation in the selected cases, partly because they share the same balancing safeguards.

Those results support the feasibility and inspectability of the workflow. They do not demonstrate that it improves grades, reliably interprets every diagram or produces universally better questions. Feedback from my family and developer friend informed practical changes but cannot stand in for a representative usability study. Model smoke tests show that the integrations have worked; they do not quantify their accuracy under varied classroom conditions.

6.3 Priorities for further work

Further work should prioritize labeled speech and image benchmarks, independently rated question quality, and consented learner studies. Task completion and unsupported-answer detection could be measured first; delayed recall needs a controlled design. Accessibility testing should include assistive technologies, while resource profiling should examine lower-powered devices, model downloads and decorative rendering.

The current result is a technically developed, testable study-support application with explicit limitations. Its defensible achievement is to make a mixed-input AI workflow usable and reviewable, while exposing where software checks stop and human judgment remains necessary. That boundary is central to the project, not an afterthought to its evaluation.

References

- [1] University of London and Goldsmiths, Final Project Ideas, CM3070 Computer Science Final Project, April 2025, section 4.1, Project Idea 1, Orchestrating AI models to achieve a goal, pp. 19-21. Distributed through the Project templates reading on Coursera. <https://www.coursera.org/learn/uol-cm3070-computer-science-final-project/supplement/hnlZI/project-templates>. Source checked for this report on 22 September 2026. Course enrolment required.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel and D. Kiela, Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020. <https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>. Consulted August-September 2026.
- [3] T. Gao, H. Yen, J. Yu and D. Chen, Enabling Large Language Models to Generate Text with Citations, Proceedings of EMNLP, pp. 6465-6488, 2023. DOI: 10.18653/v1/2023.emnlp-main.398. <https://aclanthology.org/2023.emnlp-main.398/>. Consulted August-September 2026.
- [4] S. Robertson and H. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009. DOI: 10.1561/15000000019. https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf. Consulted August-September 2026.
- [5] G. V. Cormack, C. L. A. Clarke and S. Buettcher, Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods, Proceedings of SIGIR, pp. 758-759, 2009. DOI: 10.1145/1571941.1572114. <https://cormack.uwaterloo.ca/cormacksigiro9-rrf.pdf>. Consulted August-September 2026.
- [6] J. Carbonell and J. Goldstein, The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries, Proceedings of SIGIR, pp. 335-336, 1998. DOI: 10.1145/290941.291025. https://www.cs.cmu.edu/afs/cs/Web/People/jgc/publication/MMR_DiversityBased_Reranking_SIGIR_1998.pdf. Consulted August-September 2026.
- [7] N. Reimers and I. Gurevych, Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, Proceedings of EMNLP-IJCNLP, pp. 3982-3992, 2019. DOI: 10.18653/v1/D19-1410. <https://aclanthology.org/D19-1410/>. Consulted August-September 2026.

- [8] Sentence Transformers, all-MiniLM-L6-v2 model card, undated. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Consulted August-September 2026. Browser conversion used in the project: Xenova/all-MiniLM-L6-v2.
- [9] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, Robust Speech Recognition via Large-Scale Weak Supervision, Proceedings of the 40th International Conference on Machine Learning, PMLR, vol. 202, pp. 28492-28518, 2023. <https://proceedings.mlr.press/v202/radford23a.html>. Consulted August-September 2026.
- [10] J. Li, D. Li, C. Xiong and S. Hoi, BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation, Proceedings of the 39th International Conference on Machine Learning, PMLR, vol. 162, pp. 12888-12900, 2022. <https://proceedings.mlr.press/v162/li22n.html>. Consulted August-September 2026.
- [11] B. Xiao, H. Wu, W. Xu, X. Dai, H. Hu, Y. Lu, M. Zeng, C. Liu and L. Yuan, Florence-2: Advancing a Unified Representation for a Variety of Vision Tasks, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4818-4829, 2024. https://openaccess.thecvf.com/content/CVPR2024/html/Xiao_Florence-2_Advancing_a_Unified_Representation_for_a_Variety_of_Vision_CVPR_2024_paper.html. Consulted August-September 2026.
- [12] H. L. Roediger III and J. D. Karpicke, Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention, Psychological Science, vol. 17, no. 3, pp. 249-255, 2006. DOI: 10.1111/j.1467-9280.2006.01693.x. https://learninglab.psych.purdue.edu/downloads/2006/2006_Roediger_Karpicke_PsychSci.pdf. Consulted August-September 2026.
- [13] R. E. Mayer, Multimedia Learning, 2nd ed. Cambridge University Press, 2009. <https://www.cambridge.org/core/books/multimedia-learning/7A62F072A71289E1E262980CB026A3F9>. Consulted August-September 2026.
- [14] World Wide Web Consortium, Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, 12 December 2024. <https://www.w3.org/TR/2024/REC-WCAG22-20241212/>. Consulted August-September 2026.
- [15] E. M. Bender, T. Gebru, A. McMillan-Major and S. Shmitchell, On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?, Proceedings of the ACM Conference on Fairness, Accountability, and Transparency, pp. 610-623, 2021. DOI: 10.1145/3442188.3445922. <https://doi.org/10.1145/3442188.3445922>. Consulted August-September 2026.

- [16] Hugging Face, Transformers.js documentation, software documentation, undated. <https://huggingface.co/docs/transformers.js/en/index>. Consulted August-September 2026. Implementation uses the 3.x package line recorded in package-lock.json.
- [17] Shoelace contributors, Shoelace, web-component library, version 2.20.1, MIT license. <https://github.com/shoelace-style/shoelace>. Consulted August-September 2026.
- [18] Tesseract.js contributors, Tesseract.js, browser OCR library, version 5.x. <https://github.com/naptha/tesseract.js>. Consulted August-September 2026.
- [19] F. Ali, smokey-fluid-cursor, version 1.0.7, MIT-licensed software package. <https://github.com/faraasat/smokey-fluid-cursor>. Consulted August-September 2026. Project dependency inspected 21 September 2026.
- [20] L. Zemolochinas, AI Study Companion, source code and evaluation records, 2026. <https://github.com/LukoDevelops/AIStudyCompanion>. Relevant records include docs/EVALUATION_LOG.md, docs/TESTING.md, tests/, and src/evaluation/benchmark.js. Checked for this report on 25 September 2026.
- [21] Mozilla and PDF.js contributors, PDF.js, PDF parsing and rendering library. <https://github.com/mozilla/pdf.js>. Package version is pinned by the project lockfile. Consulted August-September 2026.
- [22] M. Williamson and Mammoth contributors, Mammoth.js, DOCX conversion library. <https://github.com/mwilliamson/mammoth.js>. Package version is pinned by the project lockfile. Consulted August-September 2026.